RAPPORT E5

Surveillance et Résolution d'Incident

Application Flask — Classification d'images satellite

Champ	Détail
Candidat	Bertrand
Certification	Titre RNCP 37827 — Développeur en Intelligence Artificielle
Compétences visées	C20 (Surveillance) · C21 (Résolution d'incident)
Livrable	E5 — Cas Pratique : Débogage applicatif
Date de remise	Mars 2026

C20 Surveiller une application d'intelligence artificielle, en mobilisant des techniques de monitorage et de journalisation, dans le respect des normes de gestion des données personnelles en vigueur, afin d'alimenter la feedback loop dans une approche MLOps, et de permettre la détection automatique d'incidents.

C21 Résoudre les incidents techniques en apportant les modifications nécessaires au code de l'application et en documentant les solutions pour en garantir le fonctionnement opérationnel.

Sommaire

Introduction 3
1. Résolution de l'incident technique (C21) 3
1.1 Symptômes observés et cause racine 3
1.2 Reproduction de l'incident 3
1.3 Correctif appliqué 3
1.4 Stabilisation de la route /predict 3
2. Tests automatisés et prévention de la régression (C21) 4
2.1 Stratégie de test — FakeModel et walk-through de la route 4
2.2 Scénarios couverts — 14 tests verts 4
3. Journalisation et monitorage local (C20) 4
3.1 Politique de journalisation structurée 4
3.2 Métriques, seuils et niveaux d'alerte 4
3.3 Tableau de bord /monitoring et endpoint /health 5
4. Feedback loop et persistance SQLite (C20) 5
4.1 Principe et flux de collecte 5
4.2 Données personnelles et minimisation 5
5. CI/CD — Pipeline GitHub Actions (C21) 5
Conclusion 5
Annexe 1 — Matrice de couverture C20/C21 6
Annexe 2 — Procédure de lancement local 6
Annexe 3 — Workflow GitHub Actions 7
Appendice A — Traceback type de l'incident 8
Appendice B — Schéma et logique de la feedback loop SQLite 8

Introduction

Ce rapport documente le travail réalisé dans le cadre du livrable E5 (compétences C20 et C21) : partir d'une application Flask de classification d'images satellite existante et présentant un incident technique, identifier la cause racine, appliquer un correctif minimal et stable, puis mettre en place les mécanismes de surveillance et de feedback loop nécessaires à une démarche MLOps rigoureuse.

L'application permet à un utilisateur de déposer une image, d'obtenir une prédiction du modèle CNN parmi quatre classes (desert, forest, meadow, mountain), puis de signaler si la prédiction est correcte ou non. Le travail a été volontairement cadré autour d'un correctif minimal, démontrable en environnement local Windows, et sécurisé par une suite de tests et une pipeline GitHub Actions.

1. Résolution de l'incident technique (C21)

1.1 Symptômes observés et cause racine

L'incident se produisait systématiquement lors de l'appel à la route /predict. Lorsqu'un utilisateur déposait une image dont les dimensions différaient de 224×224 pixels, l'application tombait en erreur brute au lieu d'afficher une prédiction.

Cause racine identifiée

La fonction de prétraitement ouvrait l'image avec PIL, la convertissait en RGB et la normalisait, mais N'IMPOSAIT PAS de redimensionnement avant le passage au modèle.
Le CNN attend impérativement un tenseur de forme (None, 224, 224, 3) — défini lors de l'entraînement.
Toute image hors 224×224 générait une incompatibilité de shape fatale à l'inférence.

Le flux fautif était le suivant : (1) réception d'une image utilisateur, (2) ouverture avec PIL, (3) conversion en tableau numérique sans harmonisation de taille, (4) passage au modèle → erreur de shape à l'appel de model.predict.

1.2 Reproduction de l'incident

Le bug est reproductible avec n'importe quelle image dont les dimensions diffèrent de 224×224. Les cas testés incluent 600×600, 1024×768 et 3000×2000. Cette reproductibilité déterministe est un prérequis C21 : elle démontre que l'erreur a été comprise et localisée à la source plutôt que masquée.

```
# Avant correctif — forme transmise au modèle pour une image 600×600
# → (1, 600, 600, 3)  ≠  (None, 224, 224, 3) attendu par le CNN
# Résultat : ValueError: Input 0 of layer "sequential" is incompatible with the
layer
```

1.3 Correctif appliqué

Le correctif est volontairement minimal et ciblé. La fonction preprocess_from_pil() a été stabilisée pour garantir un prétraitement unique et systématique avant tout appel à model.predict.

```
def preprocess_from_pil(pil_img: Image.Image) -> np.ndarray:
    """Prépare une image PIL pour Keras : RGB → 224x224 → float32 → batch."""
    img = pil_img.convert("RGB")                          # 1. Conversion RGB
    img = img.resize((224, 224), Image.Resampling.LANCZOS) # 2. Redimensionnement
    img_array = np.asarray(img, dtype=np.float32) / 255.0 # 3. Normalisation [0,1]
    img_array = np.expand_dims(img_array, axis=0)          # 4. Axe batch
    return img_array  # Shape garantie : (1, 224, 224, 3)
```

En complément, le chargement du modèle a été rendu paresseux via get_model() décoré avec @lru_cache. Le modèle n'est plus importé au chargement du module, mais au premier appel. Ce choix apporte deux bénéfices : les tests n'ont plus besoin de charger TensorFlow à l'import, et le modèle peut être remplacé par un FakeModel dans les tests d'intégration.

```
@lru_cache(maxsize=1)
def get_model():
    """Charge le modèle Keras uniquement au premier besoin."""
    import keras
    return keras.saving.load_model(MODEL_PATH, compile=False)
```

1.4 Stabilisation de la route /predict

La route /predict a également été renforcée avec une chaîne de validation défensive : vérification de la présence du champ file, validation d'un nom non vide, contrôle des extensions autorisées (png, jpg, jpeg, webp), journalisation de chaque prédiction, et redirection propre vers / en cas d'échec. L'utilisateur ne se retrouve plus face à une erreur brute.

2. Tests automatisés et prévention de la régression (C21)

2.1 Stratégie de test — FakeModel et walk-through de la route

Un test unitaire sur la fonction preprocess_from_pil() seul ne suffisait pas : il fallait valider que la route Flask transmettait bien un tenseur 224×224 au modèle dans un contexte HTTP réaliste. Un FakeModel a été introduit comme remplaçant du modèle Keras dans les tests, via unittest.mock.patch sur get_model().

```
class FakeModel:
    """Modèle factice — valide la forme du tenseur reçu."""
    def predict(self, img_array, verbose=0):
        assert img_array.shape == (1, 224, 224, 3)  # Verrou anti-régression
        return np.array([[0.05, 0.10, 0.15, 0.70]], dtype=np.float32)

def test_predict_success_uses_resized_input(self):
    img = Image.new("RGB", (600, 400), color="purple")  # Image hors 224×224
    with patch.object(app_module, "get_model", return_value=FakeModel()):
        response = self.client.post("/predict", data={"file": (img_buf,
"test.jpg")})
    self.assertEqual(response.status_code, 200)  # Plus de crash
```

2.2 Scénarios couverts — 14 tests verts

#	Scénario testé	Objectif C21
1-4	Extensions autorisées / refusées (jpg, png, bmp, txt…)	Validation entrée
5	Génération de Data URL base64 depuis PIL	Interface HTML
6	preprocess_from_pil() → shape (1, 224, 224, 3)	Correctif shape
7	Tailles variées : 100×100, 500×300, 1024×768, 3000×2000	Anti-régression
8	GET / → HTTP 200	Route principale
9	POST /predict sans fichier → redirection /	Validation entrée
10	POST /predict nom vide → redirection /	Validation entrée

11	POST /predict extension invalide (.bmp) → redirection /	Validation entrée
12	POST /predict image 600×400 → HTTP 200, label affiché	Correctif intégration
13	GET /feedback → HTTP 200	Route feedback
14	POST /feedback → persistance SQLite vérifiée	Feedback loop
15 +16	/health → JSON valide · /monitoring → HTML tableau de bord	C20 monitorage
17	Config Flask : MAX_CONTENT_LENGTH et UPLOAD_FOLDER	Sécurité

Résultat : 14 tests verts

Commande : python -m pytest test_app.py -q
Résultat : 14 passed
Ce résultat est une preuve exécutable — pas un discours — que le bug est capturé.

3. Journalisation et monitorage local (C20)

3.1 Politique de journalisation structurée

Un module dédié logging_config.py structure les logs via des handlers rotatifs (RotatingFileHandler, 1 Mo max, 3 sauvegardes). Trois sorties sont gérées indépendamment pour répondre à des besoins distincts.

Fichier log	Contenu	Niveau
logs/app.log	Événements applicatifs généraux (prédictions, feedbacks, erreurs)	INFO
logs/error.log	Erreurs uniquement — facilite l'isolation des incidents	ERROR
logs/predictions.log	Prédictions et feedbacks — exploitable pour analyse MLOps	INFO

La rotation automatique évite toute croissance infinie des journaux en production locale. Le format inclut timestamp, niveau, module, fonction et numéro de ligne — permettant une localisation rapide de l'origine d'un événement.

```
formatter = logging.Formatter(
    "[%(asctime)s] %(levelname)s in %(module)s.%(funcName)s:%(lineno)d -
%(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
```

3.2 Métriques, seuils et niveaux d'alerte

Cinq métriques ont été retenues, proportionnées à l'application et directement observables sans outillage externe.

Métrique	Description	Seuil warning	Seuil critique
Accuracy feedback	Taux de prédictions correctes selon utilisateurs	< 75%	< 60%
Volume de feedback	Nombre de retours collectés (significativité stats)	≥ 20	—
Existence modèle	Présence du fichier .keras sur disque	Absent = alerte	—
Distribution classes prédites	Répartition des 4 classes prédites	Déséquilibre fort	—
Distribution corrections	Classes corrigées par les utilisateurs	Classe dominante	—

Le raisonnement derrière les seuils d'accuracy : en dessous de 75%, les retours utilisateurs signalent une dégradation perceptible du modèle. En dessous de 60%, la dégradation devient critique et justifie une analyse prioritaire ou un réentraînement. Le volume minimal de 20 feedbacks garantit que les statistiques sont statistiquement significatives avant de déclencher une alerte.

3.3 Tableau de bord /monitoring et endpoint /health

Deux mécanismes exposent les métriques. La route /health retourne un JSON synthétique (status, feedback_count, accuracy, model_exists) consommable par un outil externe. La route /monitoring affiche un tableau de bord HTML local, opérationnel en environnement Windows sans stack externe.

État	Condition	Couleur affichée
healthy	Accuracy ≥ 75% ou pas encore de feedback	Vert

warning	Accuracy entre 60% et 75% avec feedbacks	Ambre
critical	Accuracy < 60% avec feedbacks	Rouge

Le choix d'un dashboard HTML simple plutôt qu'une pile Prometheus/Grafana est volontaire et justifié : le livrable E5 est centré sur la résolution d'incident, pas sur l'infrastructure MLOps complète. Cet outillage minimal est déployable immédiatement, démontrable devant le jury, et conforme au critère C20 qui exige des outils opérationnels «a minima en environnement local».

4. Feedback loop et persistance SQLite (C20)

4.1 Principe et flux de collecte

La feedback loop permet de collecter les corrections utilisateurs et de constituer progressivement un jeu de données annoté pour le réentraînement du modèle. Le flux complet est le suivant.

1. Upload d'une image par l'utilisateur → route /predict
2. Prétraitement garanti 224×224 → inférence CNN → probabilités softmax
3. Affichage du résultat avec boutons de feedback (4 classes possibles)
4. Soumission du feedback → route /feedback (POST)
5. Enregistrement SQLite : image, prédiction, choix utilisateur, confiance, timestamp
6. Consolidation statistique → alimentée dans /health et /monitoring
7. Export futur possible en JSON via export_for_retraining() → réentraînement supervisé

Chaque enregistrement SQLite contient huit champs : image_filename, image_data_url, predicted_label, user_label, confidence_score, timestamp, is_correct (booléen calculé automatiquement) et model_version. Ce schéma répond directement à la consigne qui impose de récupérer l'image soumise, la prédiction du modèle et le feedback utilisateur.

Choix SQLite — Justification

→ Aucun service externe à installer (critique en environnement Windows local)
→ Intégration native Python — sqlite3 dans la stdlib
→ Persistance locale immédiatement démontrable en soutenance
→ Suffisance technique pour une maquette de feedback loop MLOps
→ En production : base relationnelle gérée + stockage objet pour les images (hors Data URL)

4.2 Données personnelles et minimisation

Le stockage de l'image en Data URL a été conservé pour satisfaire la consigne de feedback loop. Cependant, ce choix est encadré par des règles de minimisation conformes au RGPD : durée de conservation limitée, accès réservé au développeur/administrateur, usage exclusivement lié à l'amélioration du modèle, et anonymisation si le dispositif était enrichi. En trajectoire RGPD mature, la donnée image serait référencée par un identifiant d'objet plutôt que stockée intégralement en base.

5. CI/CD — Pipeline GitHub Actions (C21)

La pipeline GitHub Actions (.github/workflows/ci.yml) est composée de deux jobs séquentiels : lint (flake8) puis tests (pytest + couverture). Le job tests ne démarre que si le lint passe (needs: lint), ce qui permet un retour rapide en cas d'erreur de syntaxe.

Module	Lignes totales	Lignes non couvertes	Couverture
app.py	108	10	91%
database_models.py	93	25	73%
logging_config.py	39	1	97%
TOTAL	240	36	85%

Un fichier requirements-test.txt dédié exclut TensorFlow et Keras (~600 Mo) car le modèle est mocké dans les tests via FakeModel et le chargement paresseux get_model(). L'installation CI passe ainsi de plusieurs

minutes à quelques secondes. Le seuil minimal est fixé à 60% (--cov-fail-under=60) — le résultat obtenu de 85% le dépasse largement.

Garde-fou contre la régression

Si le bug de shape réapparaît, le FakeModel déclenche AssertionError (shape ≠ (1, 224, 224, 3)).
Le test échoue → la CI bloque l'intégration → le merge est impossible.
Ce n'est plus un discours théorique : c'est un verrou exécutable sur chaque PR.

Conclusion

Le travail réalisé permet de présenter un cas E5 cohérent et défendable devant le jury. L'incident principal a été identifié à la racine — un redimensionnement manquant dans le pipeline de prétraitement —, corrigé de façon minimale, puis sécurisé par 14 tests verts dont un verrou exécutable dans la CI.

Compétence	Éléments de preuve	Critère couvert
C21 — Incident	Cause racine identifiée (shape 224×224)	Identification
C21 — Incident	Reproduction déterministe (images 600×600, 1024×768…)	Reproduction
C21 — Incident	Correctif minimal dans preprocess_from_pil()	Résolution
C21 — Tests	14 tests verts, FakeModel, assertion shape	Anti-régression
C21 — CI	Pipeline GitHub Actions lint + tests + couverture 85%	Versionnement/CI
C20 — Logs	logging_config.py : 3 fichiers rotatifs structurés	Journalisation
C20 — Alertes	Seuils warning (75%) / critical (60%) sur accuracy	Alertes
C20 — Dashboard	/health JSON · /monitoring HTML local	Outils locaux
C20 — Feedback	SQLite feedbacks : image + prédiction + label utilisateur	Feedback loop MLOps
C20 — RGPD	Règles de minimisation et durée de conservation documentées	Conformité données

Annexe 1 — Matrice de couverture C20/C21

Cette annexe peut être lue seule. Elle synthétise la correspondance entre les attendus du référentiel et les éléments effectivement implémentés.

C21 — Résolution d'incident

Attendu référentiel	Couverture dans le projet
Cause du problème identifiée	Bug de shape entre images utilisateur et entrée modèle 224×224
Problème reproduit	Reproduction avec images 600×600, 1024×768, 3000×2000, etc.
Procédure de débogage documentée	Rapport principal, section 1 (cause + flux fautif + traceback)
Solution explicitée étape par étape	Redimensionnement imposé dans preprocess_from_pil()
Solution versionnée dans le dépôt Git	Correctifs concentrés dans le dossier app — PR dédiée
Tests anti-régression	test_app.py : 14 tests verts, FakeModel, assertion shape

C20 — Surveillance d'application

Attendu référentiel	Couverture dans le projet
Métriques et seuils listés	Accuracy, volume de feedback, distribution des classes (section 3.2)
Choix techniques justifiés	Logging rotatif, SQLite, dashboard HTML local (sections 3 et 4)
Outils opérationnels en local	/health JSON, /monitoring HTML, logs dans logs/ (Windows)
Journalisation intégrée au code	logging_config.py — logs prédiction, feedback, erreurs
Alertes configurées et en marche	Niveaux healthy/warning/critical basés sur seuils définis
Feedback loop alimentée	Enregistrement image + prédiction + label utilisateur (SQLite)
Documentation accessible	Format WCAG 2.1 — structure hiérarchique, contrastes, polices sans-serif
Procédure d'installation documentée	Annexe 2 — procédure complète Windows/PowerShell

Points de démonstration orale conseillés

- Montrer une image de grande taille (600×600) et expliquer pourquoi le bug se produisait avant correctif.
- Ouvrir test_app.py et montrer l'assertion assert img_array.shape == (1, 224, 224, 3) dans FakeModel.
- Lancer l'application puis visiter /monitoring pour montrer le tableau de bord en direct.
- Soumettre un feedback et montrer que SQLite incrémente les statistiques dans /health.

Annexe 2 — Procédure de lancement local (Windows / PowerShell)

Cette annexe est autonome. Elle décrit la procédure de démonstration locale complète.

Prérequis

- Windows avec PowerShell
- Python 3.11 (compatible avec TensorFlow 2.16.1)
- Dépendances installées depuis requirements.txt dans l'environnement de travail

Lancement de l'application

```
# 1. Naviguer vers le dossier de l'application
Set-Location "c:\Users\Utilisateur\Documents\Simplon -
2025\Bertrand-Debuggage-application-15092025\app"

# 2. Lancer Flask en mode debug
python app.py

# 3. Ouvrir dans le navigateur
# http://127.0.0.1:5000/           → Application principale (upload)
# http://127.0.0.1:5000/health      → État JSON du service
# http://127.0.0.1:5000/monitoring  → Tableau de bord local
```

Lancement des tests

```
# Désactiver les plugins pytest parasites (environnement Conda partagé)
$env:PYTEST_DISABLE_PLUGIN_AUTOLOAD='1'

# Exécution de la suite complète
python -m pytest test_app.py -q

# Résultat attendu
14 passed
```

Démonstration conseillée devant le jury

8. Uploader une image de taille supérieure à 224×224 (ex. photo standard 1920×1080).
9. Vérifier que la prédiction s'affiche sans erreur — la route /predict gère désormais toutes dimensions.
10. Cliquer sur un bouton de feedback (classe choisie) — page de confirmation avec ID SQLite.
11. Visiter /monitoring pour montrer la mise à jour en temps réel des statistiques de feedback.
12. Ouvrir test_app.py ligne 20 — montrer l'assertion assert img_array.shape == (1, 224, 224, 3).

Annexe 3 — Workflow GitHub Actions (CI)

Cette annexe est autonome. Elle documente la logique de CI retenue et les choix techniques associés.

Objectif de la CI

La CI a pour objectif d'empêcher l'intégration d'un changement qui casserait : (1) le correctif du bug 224×224, (2) la feedback loop SQLite, (3) les routes critiques de l'application Flask, (4) la qualité minimale du code (lint).

Architecture du workflow

Job	Outil	Rôle	Déclencheur
lint	flake8	Vérification qualité code (max-line-length 120)	Toujours
tests	pytest + pytest-cov	Tests de régression + couverture ≥ 60%	Après lint OK

Fichier requirements-test.txt dédié

TensorFlow et Keras (~600 Mo) sont exclus du fichier requirements-test.txt car le modèle est mocké dans les tests via FakeModel. L'installation CI passe ainsi de plusieurs minutes à quelques secondes.

```
# requirements-test.txt — CI uniquement (TensorFlow/Keras exclus)
flask==3.0.3
numpy==1.26.4
pillow==10.4.0
werkzeug==3.0.4
pytest==8.3.2
pytest-cov==5.0.0
flake8==7.1.1
```

Workflow complet .github/workflows/ci.yml

```
name: CI - Tests de non-regression E5

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

permissions:
  contents: read

jobs:
  lint:
    name: Lint (flake8)
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: ./app
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
```

```
          python-version: "3.11"
          cache: pip
          cache-dependency-path: app/requirements-test.txt
      - name: Install test dependencies
        run: pip install -r requirements-test.txt
      - name: Flake8
        run: flake8 --count --show-source --statistics

  tests:
    name: Tests de regression (pytest + coverage)
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements-test.txt
      - name: Run pytest with coverage
        run: |
          python -m pytest test_app.py -v --tb=short -p pytest_cov
            --cov=app --cov=database_models --cov=logging_config
            --cov-report=term-missing --cov-fail-under=60
      - uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: app/htmlcov/
          retention-days: 14
```

Résultats obtenus en local

Vérification	Résultat
flake8	0 erreur, 0 warning
pytest	14 tests passés
Couverture globale	85% (seuil minimal 60%)
Couverture app.py	91%
Couverture database_models.py	73%
Couverture logging_config.py	97%

Appendice A — Traceback type de l'incident

Cet appendice doit être lu avec le rapport principal. Il approfondit la forme concrète de l'erreur initiale.

Forme attendue par le modèle

```
# Architecture du CNN — définie à l'entraînement
Input shape: (None, 224, 224, 3)
# None = taille de batch flexible, 224x224 = dimensions spatiales FIXES, 3 = canaux
RGB
```

Forme fautive transmise avant correctif

```
# Exemple d'image utilisateur 600×600 sans redimensionnement
Forme fautive : (1, 600, 600, 3)

# Erreur résultante à l'appel de model.predict()
ValueError: Input 0 of layer "sequential" is incompatible with the layer:
  expected shape=(None, 224, 224, 3), found shape=(1, 600, 600, 3)
```

Interprétation

Le batch est correct (1) et le nombre de canaux est correct (3), mais les dimensions spatiales (600×600) ne correspondent pas aux dimensions définies lors de l'entraînement du CNN (224×224). Le modèle refuse donc l'entrée avec une ValueError — non gérée dans le code original, ce qui produisait une erreur HTTP 500 non contrôlée à l'utilisateur.

Lien avec le correctif

Le correctif consiste à garantir, avant tout appel à model.predict(), que l'image a été convertie au format RGB puis redimensionnée à exactement 224×224 avec LANCZOS (rééchantillonnage de haute qualité). La forme (1, 224, 224, 3) est alors garantie quelle que soit la taille de l'image soumise par l'utilisateur.

Appendice B — Schéma et logique de la feedback loop SQLite

Cet appendice doit être lu avec le rapport principal. Il approfondit la logique de stockage de la feedback loop.

Flux de la feedback loop

Étape	Action	Composant
1	Utilisateur uploade une image	Route /predict (POST)
2	Prétraitement garanti 224×224	preprocess_from_pil()
3	Prédiction CNN → label + confiance	get_model().predict()
4	Affichage résultat + boutons feedback	Template result.html

5	Utilisateur choisit la classe correcte	Formulaire HTML
6	Soumission feedback	Route /feedback (POST)
7	Enregistrement en base	FeedbackDatabase.save_feedback()
8	Statistiques consolidées	FeedbackDatabase.get_statistics()
9	Exposition métriques	Routes /health et /monitoring
10	Export futur pour réentraînement	export_for_retraining() → JSON

Schéma de la table feedbacks (SQLite)

Champ	Type SQLite	Rôle
id	INTEGER PK AUTOINCREMENT	Identifiant unique de l'enregistrement
image_filename	TEXT	Nom original du fichier soumis
image_data_url	TEXT	Image encodée base64 (Data URL) — stockage temporaire
predicted_label	TEXT	Classe prédite par le modèle CNN
user_label	TEXT	Classe choisie par l'utilisateur (vérité terrain)
confidence_score	REAL	Score softmax de la prédiction (∈ [0,1])
timestamp	DATETIME	Horodatage automatique de l'enregistrement
is_correct	BOOLEAN	predicted_label == user_label (calculé automatiquement)
model_version	TEXT	Version du modèle utilisé — traçabilité MLOps

Usage métier des données collectées

- Mesurer la qualité perçue du modèle sur des données réelles (vs jeu de test synthétique).
- Détecter une classe plus souvent corrigée que les autres — signe de faiblesse du modèle sur cette catégorie.
- Préparer un futur jeu de réentraînement supervisé avec images et labels certifiés humains.
- Prioriser les corrections du pipeline de données ou l'augmentation du jeu d'entraînement.

Limite connue et trajectoire production

Limite actuelle

Le stockage de l'image complète sous forme de Data URL est acceptable pour une maquette locale, mais ne constitue pas une cible de production.
En production : stockage objet (S3, Azure Blob, GCS) avec référencement par identifiant.
La base SQLite serait remplacée par PostgreSQL ou une base relationnelle gérée.
Les images seraient anonymisées ou pseudonymisées conformément au RGPD.

Rapport rédigé en conformité avec le référentiel RNCP 37827 — Développeur en Intelligence Artificielle — Livrable E5 — Compétences C20 et C21.